

UNIVERSIDADE FEDERAL DE GOIÁS

IMPLEMENTAÇÃO E ANÁLISE DE ESTRATÉGIAS DE ALGORITMOS PARA SOLUÇÃO DE PROBLEMA NP

Ciclo Hamiltoniano Direcionado

Goiânia
2026

Autores

Fabio Firanzi — [matrícula]
Finéias Antonio — [matrícula]
Heitor Vaz — [matrícula]
Tulio Vitorette — [matrícula]

Sumário

1 Introdução **3**
1.1 O problema . . . 3
1.2 Exemplos . . . 3
1.3 Aplicações práticas . . . 4

2 Classes do Ciclo Hamiltoniano Direcionado **4**
2.1 CHD ∈ NP . . . 4
2.2 CHD é NP-completo (além do que o edital pede) . . . 5

3 Implementação **5**
3.1 Estrutura comum das implementações . . . 5
3.2 Força bruta . . . 7
3.3 Backtracking . . . 8
3.4 Divisão e conquista . . . 9
3.5 Programação dinâmica (Held-Karp) . . . 10
3.6 Guloso . . . 12
3.7 Heurística: recozimento simulado . . . 15

4 Análise de complexidade **17**
4.1 Verificador . . . 17
4.2 Força bruta . . . 17
4.3 Backtracking . . . 18
4.4 Programação dinâmica (Held-Karp) . . . 18
4.5 Guloso . . . 19
4.6 Recozimento simulado . . . 19
4.7 Resumo . . . 20

5 Análise de resultados **20**
5.1 Como os testes foram feitos . . . 20
5.2 Os métodos exatos no pior caso . . . 20
5.3 Quando o digrafo tem ciclo . . . 22
5.4 Guloso e recozimento . . . 23

5.5 Qual usar em cada caso 24

6 Conclusão **24**

1 Introdução

1.1 O problema

Seja $G = (V, A)$ um grafo direcionado simples, sem laços, com $n = |V| \geq 2$ vértices. Um ciclo hamiltoniano direcionado é uma sequência $(v_1, v_2, \ldots, v_n)$ tal que

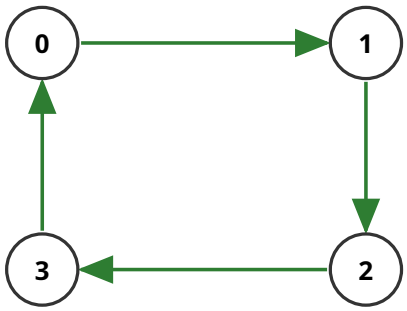
- os v_i são todos distintos e cobrem V;
- $(v_i, v_{i+1}) \in A$ para $i = 1, \ldots, n-1$;
- $(v_n, v_1) \in A$.

O problema de decisão pergunta: dado G, existe um ciclo hamiltoniano direcionado? O código deste trabalho resolve a versão de busca: devolve a sequência quando ela existe. A resposta de decisão é o sim ou o não dessa busca, e a própria sequência é a prova de que a resposta é sim, usada na Seção 2.

Convenção de simetria. Rotacionar um ciclo não cria outro ciclo. Por isso todos os métodos exatos fixam $v_1 = 0$. Inverter o sentido, ao contrário, produz outro ciclo direcionado e não é equivalente: o sentido oposto continua valendo e entra na busca.

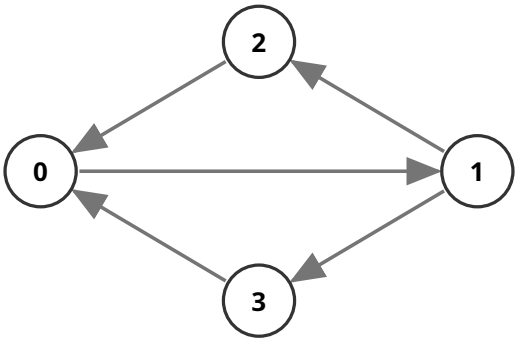
1.2 Exemplos

Exemplo com ciclo. Arcos $(0,1), (1,2), (2,3), (3,0)$. O ciclo é $0 \to 1 \to 2 \to 3 \to 0$, em verde na figura.



Digrafo com ciclo hamiltoniano direcionado.

Exemplo sem ciclo, com o digrafo forte. Arcos $(0,1), (1,2), (2,0), (1,3), (3,0)$. De 3 só se sai para 0, e em 3 só se entra por 1, então 3 teria de ocupar o trecho $1 \to 3 \to 0$. O vértice 2 não cabe nesse trecho com os arcos existentes.



Digrafo fortemente conexo sem ciclo hamiltoniano.

1.3 Aplicações práticas

Roteirização urbana. Aplicativos de entrega e frotas de caminhões usam o conceito para planejar rotas em cidades cheias de ruas de mão única, viadutos e conversões proibidas. Cada cliente é um vértice e cada trecho permitido de um ponto a outro é um arco, de modo que o digrafo é naturalmente direcionado. O objetivo é visitar todos os clientes sem repetir trajetos e voltar ao depósito, que é a volta ao início do ciclo.

Perfuração de placas de circuito (PCBs). Máquinas automatizadas precisam furar milhares de pontos em uma placa eletrônica. Cada furo é um vértice e cada deslocamento possível da broca é um arco. Encontrar o ciclo hamiltoniano de menor caminho faz a broca passar por todos os furos uma única vez, reduzindo drasticamente o tempo de produção.

Montagem de genomas (sequenciamento de DNA). Ao ler o DNA, os equipamentos quebram a fita genética em milhões de pequenos fragmentos (*reads*). Os algoritmos montam um digrafo em que os vértices são esses fragmentos e os arcos indicam sobreposições válidas. Encontrar um caminho ou ciclo hamiltoniano permite reconstruir a sequência original completa do DNA.

2 Classes do Ciclo Hamiltoniano Direcionado

2.1 CHD $\in$ NP

Um problema está em NP quando, se alguém propõe uma resposta "sim", dá para conferi-la rapidamente, em tempo polinomial [3, 9]. Não é preciso saber achar a resposta, só conferir.

No CHD, a resposta "sim" proposta é uma sequência de vértices, $c = (c_1, \ldots, c_n)$. Para conferir se ela é um ciclo hamiltoniano direcionado, basta checar três coisas:

1. A sequência tem exatamente n vértices.
2. Nenhum vértice se repete. Um vetor de n marcas resolve em n passos. Como são n vértices diferentes em n posições, nenhum vértice fica de fora.

3. Cada vértice tem arco para o próximo, e o último tem arco para o primeiro. São n consultas à matriz, cada uma em tempo constante.

Ao todo são cerca de n passos, um tempo polinomial. A conferência aceita se, e somente se, c é de fato um ciclo hamiltoniano direcionado de G. Logo, CHD está em NP.

2.2 CHD é NP-completo (além do que o edital pede)

Um problema é NP-completo quando está em NP e é pelo menos tão difícil quanto os problemas mais difíceis de NP. Para mostrar isso, usa-se um problema que já se sabe ser difícil: o ciclo hamiltoniano em grafos sem direção, que é NP-completo.

A ideia é transformar um problema no outro:

1. Pegue um grafo sem direção e troque cada aresta entre u e v por dois arcos, um de u para v e outro de v para u. Isso gera um digrafo.
2. Se o grafo tem um ciclo hamiltoniano, o digrafo tem o mesmo ciclo, pois todo caminho existe nos dois sentidos.
3. Se o digrafo tem um ciclo hamiltoniano, o grafo também tem, pois cada arco usado veio de uma aresta.

Logo, quem soubesse resolver o CHD saberia resolver o ciclo hamiltoniano sem direção, que é um problema difícil. Então o CHD também é difícil e, como já está em NP, é NP-completo.

3 Implementação

3.1 Estrutura comum das implementações

Cada estratégia é um programa independente, com a sua própria função `main`. Todos usam as mesmas peças, descritas abaixo, para que as comparações sejam justas: o digrafo, o contador e o verificador são iguais para todos.

Organização dos arquivos. Há uma pasta para a parte comum e uma para cada estratégia. Cada pasta de estratégia tem o arquivo com o algoritmo (por exemplo, `forca_bruta.cpp`) e o seu `main.cpp`, que apenas prepara o digrafo, chama o algoritmo e mostra o resultado. O comando `make` compila todos os programas.

O digrafo. É guardado como uma tabela $n \times n$, em que a posição (u, v) indica se existe o arco de u para v. Assim, perguntar se um arco existe custa um passo, o que importa porque todos os algoritmos fazem essa pergunta muitas vezes. A tabela ocupa n^2 posições. Não existem laços (arcos de um vértice para ele mesmo): se alguém tenta adicionar um, ele é ignorado. A tabela distingue os sentidos, então a existência de $u \to v$ não implica a de $v \to u$.

O contador de operações. Para a análise por contagem de instruções, o digrafo conta toda consulta de arco que recebe, e cada algoritmo conta também a sua unidade de trabalho principal. São três contadores:

- `consultas_arco`: quantas vezes se perguntou se um arco existe. Todos os algoritmos usam.
- `permutacoes`: quantas ordens de vértices foram testadas. Só a força bruta usa.
- `nos`: a unidade de trabalho dos demais algoritmos, que muda de um para outro: chamadas da busca no backtracking, estados expandidos no Held-Karp, escolhas no guloso e movimentos no recozimento.

Os números de algoritmos diferentes não são somados em um total único, porque um estado do Held-Karp e uma permutação da força bruta não são a mesma operação.

O verificador. A função `verifica_ciclo` recebe um digrafo e uma sequência de vértices e responde se ela é um ciclo hamiltoniano direcionado: confere se tem n vértices, se nenhum se repete e se cada vértice tem arco para o seguinte, inclusive do último para o primeiro. Todos os programas o usam para conferir o ciclo que o algoritmo devolveu, antes de mostrá-lo.

O gerador de digrafos. Cria os digrafos de teste a partir de um tipo e do tamanho n:

Tipo	O que gera
`ciclo_simples`	Só o ciclo $0 \to 1 \to \cdots \to (n-1) \to 0$. Tem ciclo hamiltoniano, e é o único.
`forte_sem_ciclo`	O exemplo fixo de 4 vértices da introdução: fortemente conexo e sem ciclo hamiltoniano. Ignora o n.
`pior_caso`	Um digrafo completo sobre $n-1$ vértices, mais um vértice ligado só ao vértice 1. Não tem ciclo, e obriga os métodos exatos a esgotar a busca.
`aleatorio`	Cada arco existe com probabilidade 0,35. Pode ter ou não ter ciclo, e a chance de ter cresce com n.
`armadilha_indice`	Digrafo de 4 vértices em que o guloso por índice falha, embora exista ciclo.
`armadilha_warnsdorff`	Digrafo de 4 vértices em que o guloso de Warnsdorff falha, embora exista ciclo.

O tipo `aleatorio` usa uma semente. Sem indicação, ela é sorteada a cada execução, e o programa imprime a semente usada. Com o argumento `--semente=N` o mesmo digrafo é gerado de novo, o que permite rodar dois algoritmos no mesmo digrafo e comparar as respostas.

O programa de cada estratégia. Todos funcionam do mesmo jeito: recebem o tipo e o tamanho n pela linha de comando, geram o digrafo, rodam o algoritmo e imprimem o ciclo (ou a informação de que não foi encontrado), os contadores, o tempo e a semente do digrafo. O tempo é medido com `std::chrono::steady_clock` e cobre só a execução do algoritmo, sem a geração do digrafo.

3.2 Força bruta

A força bruta testa todas as ordens possíveis de visitar os vértices. O vértice 0 fica sempre em primeiro, porque um ciclo é circular: girar a lista não cria um ciclo diferente. Sobram $(n-1)!$ ordens para os outros vértices, geradas com `std::next_permutation`.

Para cada ordem, o programa confere se existe arco de cada vértice para o próximo e do último de volta para o primeiro. Se falta algum arco, passa para a ordem seguinte. Se todos existem, achou o ciclo e para. Se as ordens acabam sem sucesso, o ciclo não existe.

Listing 1: forca_bruta.hpp

```hpp
#pragma once

#include "00-comum/digrafo.hpp"

#include <optional>
#include <vector>

namespace chd {

std::optional<std::vector<int>> forca_bruta(const Digrafo& g, Contador& contador);

}
```

Listing 2: forca_bruta.cpp

```cpp
#include "01-forca-bruta/forca_bruta.hpp"

#include <algorithm>

namespace chd {

std::optional<std::vector<int>> forca_bruta(const Digrafo& g, Contador& contador) {
  const int n = g.n();
  if (n < 2) {
    return std::nullopt;
  }

  std::vector<int> ciclo(n);
  for (int i = 0; i < n; ++i) {
    ciclo[i] = i;
  }

  do {
    contador.permutacoes++;

    bool fecha = true;
    for (int i = 0; i < n; ++i) {
      if (!g.consulta_arco(ciclo[i], ciclo[(i + 1) % n], contador)) {
        fecha = false;
        break;
      }
    }
    if (fecha) {
      return ciclo;
    }
  } while (std::next_permutation(ciclo.begin() + 1, ciclo.end()));

```

```
  return std::nullopt;
}

}
```

3.3 Backtracking

O backtracking monta o caminho aos poucos, um vértice por vez, começando pelo 0. A cada passo, tenta ir para um vértice que ainda não foi visitado e para o qual existe arco a partir do último vértice do caminho. Quando o caminho já tem todos os vértices, só falta conferir se existe o arco de volta ao 0.

Se em algum ponto não dá para continuar, ele volta atrás: desfaz o último passo e tenta o próximo vértice possível. Isso descarta de uma só vez todas as ordens que começam com um arco que não existe, o que a força bruta testaria uma a uma. Os vértices são tentados em ordem crescente de número.

No pior caso ele ainda testa um número fatorial de caminhos, mas costuma testar bem menos.

Listing 3: backtracking.hpp

```hpp
#pragma once

#include "00-comum/digrafo.hpp"

#include <optional>
#include <vector>

namespace chd {

std::optional<std::vector<int>> backtracking(const Digrafo& g, Contador& contador);

}
```

Listing 4: backtracking.cpp

```cpp
#include "02-backtracking/backtracking.hpp"

namespace chd {
namespace {

bool busca(const Digrafo& g, std::vector<int>& caminho, std::vector<char>& usado,
    Contador& contador) {
  contador.nos++;
  const int n = g.n();
  const int ultimo = caminho.back();

  if (static_cast<int>(caminho.size()) == n) {
    return g.consulta_arco(ultimo, caminho[0], contador);
  }

  for (int v = 0; v < n; ++v) {
    if (usado[v] || !g.consulta_arco(ultimo, v, contador)) {
      continue;
    }
    usado[v] = 1;
```

```
    caminho.push_back(v);
    if (busca(g, caminho, usado, contador)) {
      return true;
    }
    caminho.pop_back();
    usado[v] = 0;
  }
  return false;
}

}

std::optional<std::vector<int>> backtracking(const Digrafo& g, Contador& contador) {
  const int n = g.n();
  if (n < 2) {
    return std::nullopt;
  }

  std::vector<int> caminho = {0};
  std::vector<char> usado(n, 0);
  usado[0] = 1;

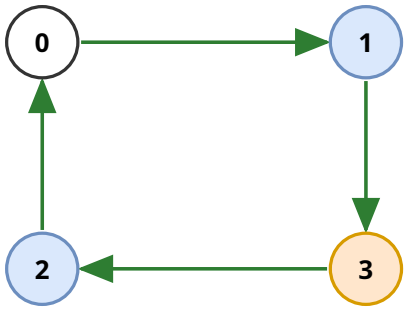
  if (busca(g, caminho, usado, contador)) {
    return caminho;
  }
  return std::nullopt;
}

}
```

3.4 Divisão e conquista

Divisão e conquista, no sentido de [3], divide o problema em partes menores que podem ser resolvidas separadamente e depois junta as respostas. No CHD isso não funciona de forma direta. Se dividirmos os vértices em dois grupos L e R e achássemos um ciclo em cada grupo, não teríamos o ciclo do digrafo inteiro, porque o ciclo certo pode passar de um grupo para o outro várias vezes.

O atalho que parece divisão é permutar os vértices de L, permutar os de R e juntar as duas listas. Contraexemplo, com origem 0, $L = \{1, 2\}$ e $R = \{3\}$: as listas possíveis são só $(0, 1, 2, 3)$, $(0, 2, 1, 3)$, $(0, 3, 1, 2)$ e $(0, 3, 2, 1)$. O digrafo de arcos $(0, 1), (1, 3), (3, 2), (2, 0)$ tem o ciclo $0 \to 1 \to 3 \to 2 \to 0$ e nenhuma das quatro listas, então esse método responde não num digrafo que tem ciclo.



O ciclo $0 \to 1 \to 3 \to 2 \to 0$ alterna entre os grupos $L = \{1, 2\}$ (azul) e $R = \{3\}$ (laranja), e nenhuma junção de listas por grupo o produz.

Por isso o grupo não apresenta esse atalho como solução.

A divisão que funciona usa outro subproblema: "existe um caminho que começa em 0, passa por um conjunto S de vértices e termina em v?". Esses subproblemas se repetem ao longo da busca. Resolver cada um uma vez e guardar a resposta é programação dinâmica, a próxima subseção. Existem também algoritmos mais recentes, como a inclusão-exclusão de Björklund para grafos sem direção [2], que ficam fora deste trabalho.

3.5 Programação dinâmica (Held-Karp)

A ideia da programação dinâmica é não refazer o mesmo trabalho. Dois caminhos que saem do 0, passam pelos mesmos vértices (em ordens diferentes) e terminam no mesmo vértice têm o mesmo futuro, pois o que falta visitar é igual nos dois. Em vez de guardar os caminhos, guarda-se só uma resposta para cada situação [6, 1].

Uma situação é descrita por dois dados: o conjunto dos vértices já visitados e o vértice em que o caminho está agora. Para cada situação, a tabela guarda sim ou não: existe um caminho que sai do 0, passa exatamente pelos vértices do conjunto e termina nesse vértice? Cada conjunto é um número binário, em que o bit de um vértice vale 1 se ele já foi visitado.

Como a tabela é preenchida.

1. Começa com uma única situação verdadeira: o conjunto só com o 0, estando no 0.
2. Os conjuntos são percorridos do menor para o maior. Para cada situação verdadeira, olha-se cada vértice u ainda não visitado para o qual existe arco a partir do vértice atual. Marca-se como verdadeira a situação "o conjunto mais o u, estando em u".
3. No fim, olha-se o conjunto com todos os vértices. Se alguma situação verdadeira termina num vértice que tem arco de volta ao 0, existe ciclo.

Como o ciclo é montado. Para devolver o ciclo, e não só sim ou não, anda-se para trás. A partir da situação final, procura-se o vértice anterior, isto é, uma situação verdadeira cujo vértice tenha arco para o vértice atual, e repete-se até chegar ao 0. A lista obtida, invertida, é o ciclo.

Exemplo. No digrafo `forte_sem_ciclo`, a tabela chega aos conjuntos $\{0, 1, 2\}$ e $\{0, 1, 3\}$ e não passa deles, porque de 2 não há arco para o 3 e de 3 não há arco para o 2. O conjunto com todos os vértices nunca é alcançado, então a resposta é que não existe ciclo.

Listing 5: held_karp.hpp

```hpp
#pragma once

#include "00-comum/digrafo.hpp"

#include <optional>
#include <vector>

namespace chd {

std::optional<std::vector<int>> held_karp(const Digrafo& g, Contador& contador);

}
```

Listing 6: held_karp.cpp

```cpp
#include "04-programacao-dinamica/held_karp.hpp"

#include <algorithm>
#include <cstddef>
#include <cstdint>

namespace chd {

std::optional<std::vector<int>> held_karp(const Digrafo& g, Contador& contador) {
  const int n = g.n();
  if (n < 2) {
    return std::nullopt;
  }

  const uint64_t limite = 1ull << n;
  std::vector<char> alcanca(static_cast<size_t>(limite) * n, 0);
  auto posicao = [n](uint64_t mascara, int v) { return static_cast<size_t>(mascara) *
    n + v; };

  alcanca[posicao(1u, 0)] = 1;

  for (uint64_t mascara = 1; mascara < limite; mascara += 2) {
    for (int v = 0; v < n; ++v) {
      if (!alcanca[posicao(mascara, v)]) {
        continue;
      }
      contador.nos++;
      for (int u = 0; u < n; ++u) {
        if ((mascara & (1ull << u)) != 0 || !g.consulta_arco(v, u, contador)) {
          continue;
        }
        alcanca[posicao(mascara | (1ull << u), u)] = 1;
      }
    }
  }

  const uint64_t cheia = limite - 1;
  int fim = -1;
```

```
38    for (int v = 1; v < n; ++v) {
39      if (alcanca[posicao(cheia, v)] && g.consulta_arco(v, 0, contador)) {
40        fim = v;
41        break;
42      }
43    }
44    if (fim < 0) {
45      return std::nullopt;
46    }
47
48    std::vector<int> ciclo;
49    uint64_t mascara = cheia;
50    int v = fim;
51    while (v != 0) {
52      ciclo.push_back(v);
53      const uint64_t anterior = mascara ^ (1ull << v);
54      int u = 0;
55      while (!(((anterior >> u) & 1ull) && alcanca[posicao(anterior, u)] && g.
        consulta_arco(u, v, contador))) {
56        ++u;
57      }
58      mascara = anterior;
59      v = u;
60    }
61    ciclo.push_back(0);
62    std::reverse(ciclo.begin(), ciclo.end());
63    return ciclo;
64  }
65
66  }
```

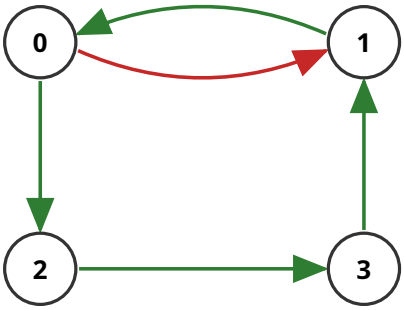
3.6 Guloso

O guloso monta um caminho a partir do 0 e nunca volta atrás. A cada passo escolhe um vértice ainda não visitado para o qual existe arco e segue em frente. Se chega a um vértice sem saída antes de visitar todos, ou se o último vértice não tem arco de volta ao 0, desiste. Há dois critérios de escolha:

- `indice`: escolhe o vértice de menor número.
- `warnsdorff`: escolhe o vértice que tem menos saídas para vértices ainda não visitados. A ideia é passar primeiro pelos vértices mais difíceis de sair.

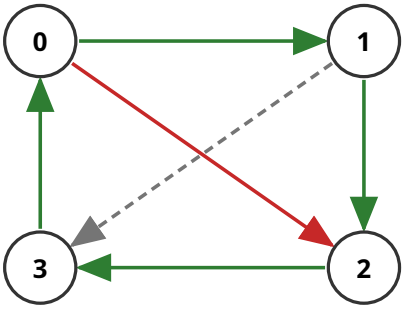
Nenhum dos dois tem garantia. Se o guloso acha um ciclo, ele está certo, mas se desiste, o ciclo pode existir. Cada critério tem um digrafo em que falha:

Critério `indice`. Arcos $(0,1),(0,2),(2,3),(3,1),(1,0)$. O ciclo $0 \to 2 \to 3 \to 1 \to 0$ existe (verde). O guloso sai de 0 para 1 (vermelho), porque 1 tem menor número que 2, e de 1 só há arco para 0, que já foi visitado. Ele desiste, embora o ciclo exista.



Armadilha do critério por índice. Verde: ciclo existente. Vermelho: escolha do guloso.

Critério `warnsdorff`. Arcos $(0,1),(0,2),(1,2),(1,3),(2,3),(3,0)$. O ciclo $0 \to 1 \to 2 \to 3 \to 0$ existe (verde). Saindo de 0, o vértice 1 tem duas saídas livres e o 2 tem uma, então a regra escolhe o 2 (vermelho). Dele segue ao 3 e desiste, com o 1 sem visitar.



Armadilha do critério de Warnsdorff. Verde: ciclo existente. Vermelho: escolha do guloso. Tracejado: arco não usado.

Listing 7: guloso.hpp

```hpp
#pragma once

#include "00-comum/digrafo.hpp"

#include <optional>
#include <vector>

namespace chd {

enum class Criterio { Indice, Warnsdorff };

std::optional<std::vector<int>> guloso(const Digrafo& g, Contador& contador, Criterio
    criterio);

}
```

Listing 8: guloso.cpp

```cpp
#include "05-guloso/guloso.hpp"

```

```cpp
#include <climits>

namespace chd {
namespace {

int saidas_livres(const Digrafo& g, int v, const std::vector<char>& usado, Contador& contador) {
  int livres = 0;
  for (int w = 0; w < g.n(); ++w) {
    if (!usado[w] && g.consulta_arco(v, w, contador)) {
      livres++;
    }
  }
  return livres;
}

}

std::optional<std::vector<int>> guloso(const Digrafo& g, Contador& contador, Criterio criterio) {
  const int n = g.n();
  if (n < 2) {
    return std::nullopt;
  }

  std::vector<int> caminho = {0};
  std::vector<char> usado(n, 0);
  usado[0] = 1;

  while (static_cast<int>(caminho.size()) < n) {
    contador.nos++;
    const int ultimo = caminho.back();

    int escolhido = -1;
    int menor = INT_MAX;
    for (int v = 0; v < n; ++v) {
      if (usado[v] || !g.consulta_arco(ultimo, v, contador)) {
        continue;
      }
      if (criterio == Criterio::Indice) {
        escolhido = v;
        break;
      }
      const int livres = saidas_livres(g, v, usado, contador);
      if (livres < menor) {
        menor = livres;
        escolhido = v;
      }
    }

    if (escolhido < 0) {
      return std::nullopt;
    }
    usado[escolhido] = 1;
    caminho.push_back(escolhido);
  }

  if (!g.consulta_arco(caminho.back(), 0, contador)) {
```

```
    return std::nullopt;
  }
  return caminho;
}

}
```

3.7 Heurística: recozimento simulado

O recozimento simulado [8] faz o contrário do guloso: começa com uma ordem completa dos vértices, mesmo que esteja errada, e a melhora aos poucos.

Cada ordem recebe uma nota: o número de arcos do ciclo que existem de fato, de 0 a n. A ordem é um ciclo hamiltoniano exatamente quando a nota é n. Essa nota não é uma garantia de qualidade, é só um guia para a busca.

O programa repete os seguintes passos:

1. Faz uma mudança pequena: tira um vértice do lugar e o coloca em outra posição.
2. Calcula a nota da nova ordem.
3. Se a nota não piorou, aceita a mudança.
4. Se piorou, às vezes aceita mesmo assim, com chance $e^{\Delta/T}$, em que Δ é a piora (um número negativo) e T é a temperatura. Aceitar uma piora de vez em quando permite sair de uma ordem ruim da qual toda mudança pequena parece pior.
5. Reduz a temperatura: $T \leftarrow 0{,}995\,T$. Quanto mais baixa, menos piora é aceita.

Ele para quando a nota chega a n (achou o ciclo) ou quando o limite de tentativas acaba. O limite é de 10 recomeços, cada um com uma ordem embaralhada e $200 \cdot n$ mudanças, com temperatura inicial $T = 1$. Esses valores foram escolhidos antes dos testes e não foram alterados.

Se o limite acaba sem achar, isso não prova que o ciclo não existe: pode ter faltado sorte ou tentativas.

Listing 9: recozimento.hpp

```
#pragma once

#include "00-comum/digrafo.hpp"

#include <optional>
#include <vector>

namespace chd {

struct Parametros {
  double temperatura_inicial = 1.0;
  double resfriamento = 0.995;
  int movimentos_por_vertice = 200;
  int recomecos = 10;
};

```

```cpp
struct ResultadoRecozimento {
  std::optional<std::vector<int>> ciclo;
  int melhor_nota = 0;
};

ResultadoRecozimento recozimento(const Digrafo& g, Contador& contador, unsigned semente,
                                 const Parametros& parametros = {});

}
```

Listing 10: recozimento.cpp

```cpp
#include "06-heuristica/recozimento.hpp"

#include <algorithm>
#include <cmath>
#include <numeric>
#include <random>

namespace chd {
namespace {

int nota(const Digrafo& g, const std::vector<int>& ordem, Contador& contador) {
  const int n = static_cast<int>(ordem.size());
  int arcos = 0;
  for (int i = 0; i < n; ++i) {
    if (g.consulta_arco(ordem[i], ordem[(i + 1) % n], contador)) {
      arcos++;
    }
  }
  return arcos;
}

}

ResultadoRecozimento recozimento(const Digrafo& g, Contador& contador, unsigned semente,
                                 const Parametros& parametros) {
  ResultadoRecozimento resultado;
  const int n = g.n();
  if (n < 2) {
    return resultado;
  }

  std::mt19937 rng(semente);
  std::uniform_int_distribution<int> sorteia_posicao(1, n - 1);
  std::uniform_real_distribution<double> sorteia_chance(0.0, 1.0);

  std::vector<int> ordem(n);
  std::iota(ordem.begin(), ordem.end(), 0);

  for (int recomeco = 0; recomeco < parametros.recomecos; ++recomeco) {
    std::shuffle(ordem.begin() + 1, ordem.end(), rng);
    int atual = nota(g, ordem, contador);
    resultado.melhor_nota = std::max(resultado.melhor_nota, atual);
    if (atual == n) {
      resultado.ciclo = ordem;
```

```
45        return resultado;
46      }
47
48      double temperatura = parametros.temperatura_inicial;
49      const int movimentos = parametros.movimentos_por_vertice * n;
50      for (int passo = 0; passo < movimentos; ++passo) {
51        contador.nos++;
52        const int origem = sorteia_posicao(rng);
53        const int destino = sorteia_posicao(rng);
54
55        std::vector<int> nova = ordem;
56        const int vertice = nova[origem];
57        nova.erase(nova.begin() + origem);
58        nova.insert(nova.begin() + destino, vertice);
59
60        const int nova_nota = nota(g, nova, contador);
61        const int variacao = nova_nota - atual;
62        if (variacao >= 0 || sorteia_chance(rng) < std::exp(variacao / temperatura)) {
63          ordem = nova;
64          atual = nova_nota;
65          resultado.melhor_nota = std::max(resultado.melhor_nota, atual);
66          if (atual == n) {
67            resultado.ciclo = ordem;
68            return resultado;
69          }
70        }
71        temperatura *= parametros.resfriamento;
72      }
73    }
74    return resultado;
75  }
76
77  }
```

4 Análise de complexidade

4.1 Verificador

Tempo. Confere o tamanho da sequência, se algum vértice se repete (com uma lista de n marcas) e se cada vértice tem arco para o seguinte (n consultas).

- $\Omega(1)$: melhor caso: o tamanho está errado e ele rejeita logo.
- $O(n)$: pior caso: cerca de n passos.
- $\Theta(n)$: quando a sequência é um ciclo, pois ele precisa conferir tudo.

Espaço. $O(n)$, pela lista de marcas.

4.2 Força bruta

Tempo. São $(n-1)!$ ordens, e cada uma consulta até n arcos.

- $\Omega(n)$: melhor caso: a primeira ordem já é o ciclo.
- $O(n!)$: pior caso: todas as ordens são testadas, e $n \cdot (n-1)! = n!$.
- $\Theta(n!)$: digrafos sem ciclo, como o `pior_caso`, porque só se afirma que não há ciclo depois de testar todas.

Espaço. $\Theta(n)$: só a lista com a ordem atual.

4.3 Backtracking

Tempo. A busca forma uma árvore com no máximo da ordem de $(n-1)!$ nós, e cada nó custa $O(n)$.

- $\Omega(n)$: melhor caso: a busca desce direto até o ciclo.
- $O(n!)$: pior caso: $(n-1)!$ nós, cada um custando n.
- $\Theta(n!)$: num digrafo completo em que nenhum arco chega ao 0: todo caminho parcial pode ser estendido até o fim, e só no último passo falta o arco de volta.

A poda abandona caminhos que não continuam e economiza por uma constante, mas não muda a ordem do pior caso.

Espaço. $\Theta(n)$: a pilha tem no máximo n níveis.

4.4 Programação dinâmica (Held-Karp)

Tempo. A tabela tem no máximo $n \cdot 2^n$ situações (conjunto de vértices visitados e vértice atual), e para cada situação verdadeira o programa olha n vértices seguintes.

- $\Omega(n\,2^n)$: melhor caso: o programa percorre a tabela inteira mesmo quando quase nada é alcançável.
- $O(n^2\,2^n)$: pior caso: quase todas as situações são verdadeiras, e cada uma gasta n consultas.
- $\Theta(n^2\,2^n)$: digrafos densos sem ciclo, como o `pior_caso`.

Montar o ciclo no fim custa $O(n^2)$, pouco perto do resto.

Espaço. $\Theta(n\,2^n)$: um byte por situação, e a tabela é criada inteira. Para $n = 22$ são cerca de 92 milhões de bytes (90 MB), e a cada vértice a mais ela dobra. A memória é o limite do método, e o programa avisa quando ela acaba, sem responder que não há ciclo.

Comparação com a força bruta. A tabela mostra o tamanho dos dois custos. Para $n = 10$ são parecidos, e a partir de $n = 15$ a diferença cresce muito: em $n = 22$ são dez ordens de grandeza.

n	$(n-1)!$	$n^2\,2^n$
10	$3{,}6 \times 10^5$	$1{,}0 \times 10^5$
15	$8{,}7 \times 10^{10}$	$7{,}4 \times 10^6$
20	$1{,}2 \times 10^{17}$	$4{,}2 \times 10^8$
22	$5{,}1 \times 10^{19}$	$2{,}0 \times 10^9$

4.5 Guloso

Tempo. São $n-1$ passos, e cada passo percorre os n vértices. No critério por índice, o passo escolhe o primeiro candidato que encontra:

- $\Omega(n)$: melhor caso: ele desiste no primeiro passo.
- $O(n^2)$: pior caso: n passos de custo n.
- $\Theta(n^2)$: no `ciclo_simples`, em que a busca passa por vértices já visitados a cada passo.

No critério de Warnsdorff, cada candidato conta suas saídas, o que custa $O(n)$ por candidato, com até n candidatos por passo:

- $\Omega(n)$: melhor caso: igual ao critério por índice.
- $O(n^3)$: pior caso: n passos de custo n^2.
- $\Theta(n^3)$: em digrafos densos, com muitos candidatos a cada passo.

Espaço. $\Theta(n)$ nos dois: o caminho e as marcas de vértices visitados.

4.6 Recozimento simulado

Tempo. Cada movimento copia a ordem, move um vértice e calcula a nota, com n consultas, e custa $\Theta(n)$. O limite é de 10 recomeços de $200 \cdot n$ movimentos, ou seja, $2000 \cdot n$ movimentos.

- $\Omega(n)$: melhor caso: a primeira ordem embaralhada já é um ciclo.
- $O(n^2)$: pior caso: usa todo o limite, $2000\,n$ movimentos de custo n.
- $\Theta(n^2)$: sempre que ele não acha o ciclo, pois então gasta todo o limite.

Espaço. $\Theta(n)$: a ordem atual e a cópia usada em cada tentativa.

4.7 Resumo

Algoritmo	Ω (melhor caso)	O (pior caso)	Θ (pior caso)	Espaço
Verificador	$\Omega(1)$	$O(n)$	$\Theta(n)$	$O(n)$
Força bruta	$\Omega(n)$	$O(n!)$	$\Theta(n!)$	$\Theta(n)$
Backtracking	$\Omega(n)$	$O(n!)$	$\Theta(n!)$	$\Theta(n)$
Held-Karp	$\Omega(n\,2^n)$	$O(n^2\,2^n)$	$\Theta(n^2\,2^n)$	$\Theta(n\,2^n)$
Guloso por índice	$\Omega(n)$	$O(n^2)$	$\Theta(n^2)$	$\Theta(n)$
Guloso de Warnsdorff	$\Omega(n)$	$O(n^3)$	$\Theta(n^3)$	$\Theta(n)$
Recozimento	$\Omega(n)$	$O(n^2)$	$\Theta(n^2)$	$\Theta(n)$

A força bruta, o backtracking e o Held-Karp têm resposta sempre certa. O guloso e o recozimento não têm garantia: se acham um ciclo ele está certo, mas se não acham, o ciclo pode existir.

5 Análise de resultados

5.1 Como os testes foram feitos

Os programas foram compilados com o g++ na opção `-O2`. Cada programa mede o tempo só do algoritmo e conta os passos, como descrito na Seção 3.1. Foram usados estes tipos de digrafo:

- `pior_caso`: sem ciclo, e sempre o mesmo digrafo para um dado n. O tempo é a mediana de 5 execuções.
- `aleatorio`: cada arco existe com probabilidade 0,35. Cada digrafo é identificado por uma semente, então dois programas podem rodar no mesmo digrafo. O Held-Karp serve de juiz, porque sua resposta é sempre certa.
- `ciclo_simples` e as duas armadilhas do guloso.

5.2 Os métodos exatos no pior caso

O tipo `pior_caso` não tem ciclo e obriga os métodos exatos a procurar em tudo. A tabela mostra o resultado, e a Figura 1 e a Figura 2 mostram os mesmos dados em escala logarítmica.

	Força bruta		Backtracking		Held-Karp	
n	passos	ms	passos	ms	passos	ms
6	120	0.002	81	0.002	41	0.002
7	720	0.006	391	0.007	97	0.004
8	5 040	0.048	2 283	0.029	225	0.008
9	40 320	0.317	15 657	0.164	513	0.020
10	362 880	3.192	123 301	1.331	1 153	0.044
11	3 628 800	34.7	1 096 011	13.2	2 561	0.120
12	39 916 800	403.1	10 850 511	132.5	5 633	0.229
13	–	–	118 369 213	1468	12 289	0.503
14	–	–	–	–	26 625	1.136
15	–	–	–	–	57 345	2.511
16	–	–	–	–	122 881	5.393
17	–	–	–	–	262 145	11.9
18	–	–	–	–	557 057	25.9
19	–	–	–	–	1 179 649	55.9
20	–	–	–	–	2 490 369	120.5

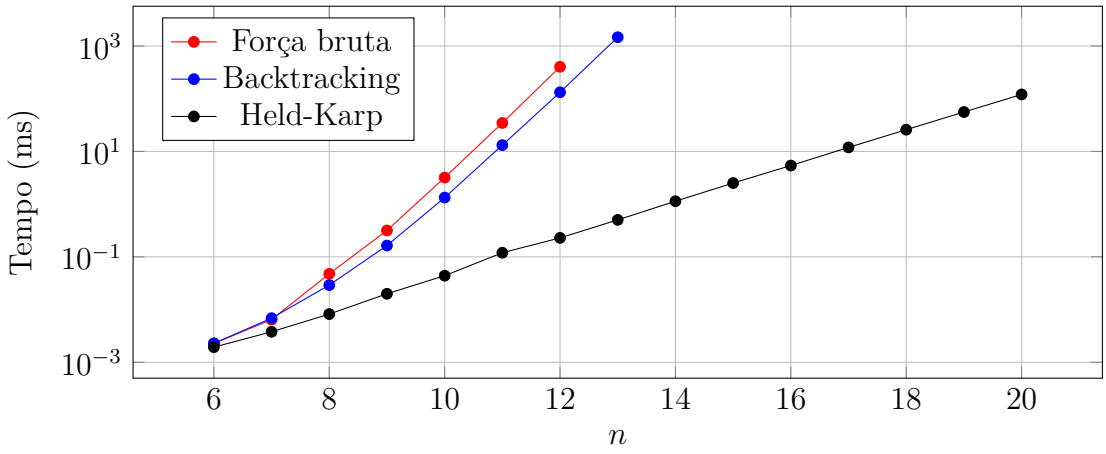


Figura 1: Tempo contra n no tipo `pior_caso`, em escala logarítmica.

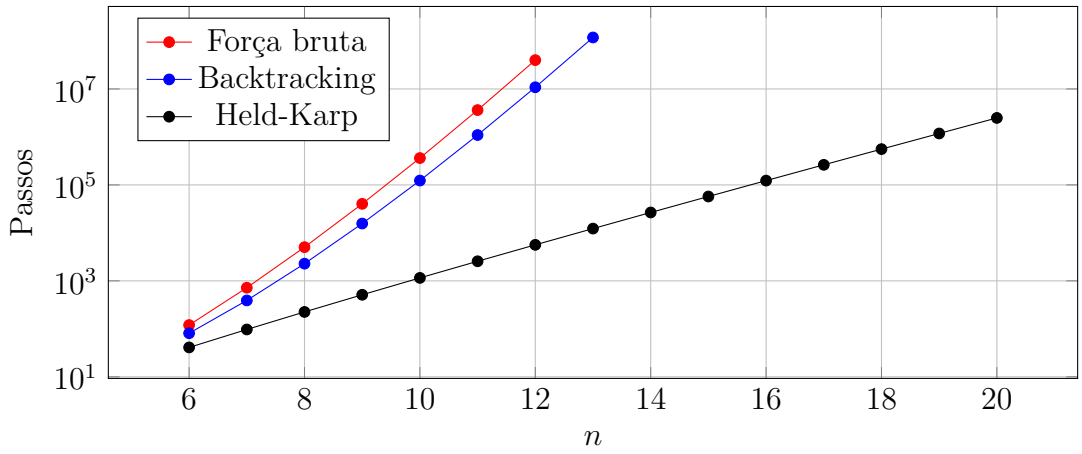


Figura 2: Passos contra n no tipo `pior_caso`, em escala logarítmica.

Força bruta. Os passos medidos são exatamente $(n-1)!$: 120 para $n = 6$, 3,6 milhões para $n = 11$ e 39,9 milhões para $n = 12$. É o que a análise de complexidade previa. O tempo cresce cerca de 12 vezes ao passar de $n = 11$ para $n = 12$ (34,7 ms para 403 ms), parecido com o fator $n - 1 = 11$.

Backtracking. Faz cerca de um terço dos passos da força bruta: 1,1 milhão contra 3,6 milhões em $n = 11$. Isso o torna mais rápido a partir de $n = 8$, mas não muda como o tempo cresce. Ele continua multiplicando por cerca de 10 a cada vértice a mais (13,2 ms em $n = 11$, 132 ms em $n = 12$ e 1,5 segundo em $n = 13$). A economia de passos é uma constante, e o crescimento continua fatorial.

Held-Karp. Os passos e o tempo crescem cerca de 2,2 vezes a cada vértice a mais, como esperado de um custo da ordem de 2^n: os passos vão de 5 633 para 12 289 e depois 26 625, e o tempo de 0,23 ms para 0,50 e 1,1 ms. Ele chega a $n = 20$ em 120 ms. Em $n = 11$ é cerca de 290 vezes mais rápido que a força bruta (0,12 ms contra 34,7 ms), e em $n = 12$ passa de 1 700 vezes (0,23 ms contra 403 ms). Para n pequeno (6 e 7) os três empatam, dentro do ruído, e a vantagem do Held-Karp aparece a partir de $n = 8$.

5.3 Quando o digrafo tem ciclo

O `pior_caso` é o pior cenário. O tipo `aleatorio` mostra o outro lado. Para cada n foram gerados 30 digrafos (sementes de 1 a 30), e o backtracking e o Held-Karp rodaram nos mesmos digrafos. A tabela mostra a mediana nos digrafos que têm ciclo.

		Backtracking		Held-Karp	
n	com ciclo	passos	ms	passos	ms
10	15 de 30	45	0.003	446	0.026
12	20 de 30	192	0.008	2 567	0.165
14	28 de 30	157	0.008	13 810	0.785
16	29 de 30	517	0.018	80 198	4.566
18	29 de 30	348	0.014	496 932	27.4

Aqui a situação se inverte. O backtracking visita poucas centenas de nós, e esse número quase não cresce com n: 45, 192, 158, 517 e 348. Como o digrafo tem muitos arcos, existem muitos ciclos, e a busca chega a um deles sem explorar quase nada. O Held-Karp, ao contrário, sempre preenche a tabela, e seus passos dobram a cada vértice: em $n = 18$ ele expande 496 932 situações contra 348 nós do backtracking, cerca de 1 400 vezes mais. O tempo acompanha: 27 ms para o Held-Karp e menos de 0,02 ms, abaixo do que se consegue medir, para o backtracking.

Nos digrafos sem ciclo o resultado muda:

n	sem ciclo	Backtracking (ms)	Held-Karp (ms)
10	15 de 30	0.006	0.013
12	10 de 30	0.045	0.065
14	2 de 30	0.944	0.458
16	1 de 30	14.4	2.539
18	1 de 30	365.1	13.2

Para n pequeno os dois são parecidos. Nos maiores há poucos digrafos sem ciclo (um em $n = 16$ e um em $n = 18$), então os números são só um indício, mas eles mostram o risco do backtracking: no digrafo de $n = 18$ ele levou 365 ms, contra 13 ms do Held-Karp. Sem ciclo, o backtracking precisa esgotar a busca, e o tempo depende de quanto da árvore sobrevive até o fim. O Held-Karp tem custo previsível, que quase não depende do digrafo.

5.4 Guloso e recozimento

Esses dois métodos não garantem a resposta, então a pergunta é quantos ciclos eles acham. Para os digrafos `aleatorio` que têm ciclo (segundo o Held-Karp), a tabela mostra em quantos cada método achou o ciclo. Foram usados 200 digrafos para $n = 10$ e $n = 14$, e 100 para $n = 18$.

n	Digrafos com ciclo	Guloso por índice	Guloso de Warnsdorff	Recozimento
10	110 de 200	7 de 110	11 de 110	110 de 110
14	178 de 200	7 de 178	9 de 178	177 de 178
18	98 de 100	2 de 98	7 de 98	98 de 98

O guloso acha poucos: no máximo 11 de 110 digrafos com ciclo. O recozimento achou todos, exceto um em $n = 14$, e esse mesmo digrafo foi resolvido ao repetir o recozimento com outra semente. Quando o recozimento diz que não achou, o Held-Karp confirmou que não há ciclo em quase todos os casos, mas isso é uma tendência e não uma garantia.

Em $n = 50$ o Held-Karp não roda, porque a tabela teria cerca de 56 petabytes. Sem juiz, nos 100 digrafos testados o recozimento achou o ciclo em todos (100 de 100), e cada ciclo achado prova que ele existe. O guloso achou em 4 (14 com a regra de Warnsdorff).

Onde o recozimento sofre. No tipo `ciclo_simples` o único ciclo possível é $0 \to 1 \to \cdots \to (n-1) \to 0$. O guloso acha na hora, porque cada vértice tem uma só saída e não há o que escolher. O recozimento vai mal:

n	Guloso (os dois critérios)	Recozimento, em 20 sementes
8	achou	20 de 20
12	achou	20 de 20
16	achou	20 de 20
20	achou	20 de 20
25	achou	16 de 20
30	achou	13 de 20
40	achou	0 de 20

Como só uma ordem entre $(n-1)!$ é o ciclo, a nota quase não ajuda: uma ordem com quase todos os arcos certos não parece melhor do que uma ordem sem sentido. Por isso a busca anda às cegas, e a chance de achar cai com n.

As armadilhas. Nos dois digrafos `armadilha_indice` e `armadilha_warnsdorff`, cada critério do guloso falha numa delas, mesmo havendo ciclo. O backtracking, o Held-Karp, a força bruta e o recozimento acharam o ciclo nas duas.

5.5 Qual usar em cada caso

- **Força bruta:** só para n muito pequeno (até cerca de 9), ou como referência simples para conferir os outros.
- **Backtracking:** quando o digrafo tem muitos arcos e é provável que exista ciclo. Acha em poucos passos, mas pode demorar muito se não houver ciclo.
- **Held-Karp:** quando se precisa de uma resposta certa com tempo previsível, inclusive para dizer que não há ciclo. O limite é a memória, que dobra a cada vértice.
- **Guloso:** quando cada vértice tem poucas saídas e há pouca escolha. É o mais rápido, mas erra com facilidade.
- **Recozimento:** quando n é grande demais para os exatos e o digrafo tem muitas saídas. Achou quase tudo nos testes, mas vai mal quando só existe um ciclo possível e nunca prova que o ciclo não existe.

Os métodos exatos garantem a resposta, tanto o "sim" quanto o "não". O guloso e o recozimento garantem só o "sim": se acharem um ciclo, ele está certo.

6 Conclusão

O trabalho implementa, com contagem de instruções e medição de tempo, cinco das seis estratégias pedidas (força bruta, backtracking, programação dinâmica, guloso e recozimento como heurística) e justifica por escrito por que divisão e conquista não se sustenta no caso geral. O resultado experimental principal é a separação por ordem de grandeza: força bruta cresce com $(n-1)!$ nós medidos, backtracking diminui a constante e continua fatorial, e Held-Karp cresce com 2^n, com razão de cerca de 800 vezes em $n = 11$.

Limites. O teto prático do Held-Karp é a memória, não o tempo: a tabela de predecessores chega a cerca de 385 MB em $n = 22$. O guloso não tem garantia e erra nos contraexemplos da Seção 3.6. O recozimento recupera quase todos os ciclos nos testes, mas nunca prova inexistência, e por isso nunca decreta "não existe": ele separa "ciclo válido" de "orçamento esgotado". Os tempos abaixo de 0,005 ms foram descartados por resolução do relógio, e o tipo de limiar teve poucos digrafos com ciclo.

Dificuldades. (i) A reconstrução do ciclo no Held-Karp exige guardar o predecessor de cada estado, o que multiplica a memória por n. (ii) A tentação de apresentar o particionamento em blocos como divisão e conquista; o contraexemplo mostrou o falso negativo, e o trabalho o expõe em vez de o esconder. (iii) O tipo aleatório ingênuo dá digrafos ou triviais (DAG) ou fáceis (arcos extras), e a comparação entre os exatos só ficou informativa com o tipo pior caso. (iv) Distinguir, em todos os métodos, "não encontrou" de "não existe", para não afirmar o que a heurística não prova.

Referências

[1] R. Bellman. Dynamic programming treatment of the travelling salesman problem. Journal of the ACM, 9(1):61–63, 1962.

[2] A. Björklund. Determinant sums for undirected Hamiltonicity. In Proc. 51st IEEE FOCS, 2010.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. Introduction to Algorithms. 3. ed., MIT Press/McGraw-Hill, 2009.

[4] A. M. Frieze. An algorithm for finding Hamilton cycles in random directed graphs. Journal of Algorithms, 9(2):181–204, 1988.

[5] M. R. Garey, D. S. Johnson. Computers and Intractability: A Guide to the Theory of NP-Completeness. W. H. Freeman, 1979.

[6] M. Held, R. M. Karp. A dynamic programming approach to sequencing problems. Journal of the SIAM, 10(1):196–210, 1962.

[7] R. M. Karp. Reducibility among combinatorial problems. In Complexity of Computer Computations, Plenum, 1972, pp. 85–103.

[8] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi. Optimization by simulated annealing. Science, 220(4598):671–680, 1983.

[9] N. Ziviani. Projeto de Algoritmos com implementações em Java e C++. Cengage Learning (Thomson/Pioneira), São Paulo, 1. ed., 2006.